

Université de Aïn Témouchent

Département de Mathématiques et Informatique

Master 1 : Cybersécurité et Intelligence Artificielle

TP : Cryptographie Asymétrique et Échange de Clés Sécurisé

Objectifs pédagogiques

- Comprendre les principes fondamentaux de la cryptographie asymétrique ;
- Comparer les performances entre cryptographie asymétrique et symétrique ;
- Mettre en œuvre des échanges de clés sécurisés (Diffie–Hellman, ECDH) ;
- Construire un schéma de chiffrement hybride ;
- Illustrer des vulnérabilités historiques (Parity Oracle, Puzzles de Merkle).

1 Exercice 1 – Échange de clé Diffie–Hellman

```
import secrets, hashlib
from Crypto.Cipher import AES

p = 0xFFFFFFFFB # petit nombre premier (TP uniquement)
g = 5

a = secrets.randbelow(p)
b = secrets.randbelow(p)

A = pow(g, a, p)
B = pow(g, b, p)

K_M = pow(B, a, p)
K_A = pow(A, b, p)
print("Clé commune identique ?", K_M == K_A)

key = hashlib.sha256(str(K_M).encode()).digest()
aes = AES.new(key, AES.MODE_EAX)
ciphertext, tag = aes.encrypt_and_digest(b"Message secret partage")
print("Message chiffré :", ciphertext)
```

2 Exercice 2 – RSA

```
from sympy import randprime, mod_inverse
from time import time

bits = 1024
p = randprime(2**(bits-1), 2**bits)
q = randprime(2**(bits-1), 2**bits)
```

```
n = p * q
phi = (p-1)*(q-1)
e = 65537
d = mod_inverse(e, phi)

print("Taille de la clé RSA :", n.bit_length(), "bits")

msg = int.from_bytes(b"RSA_Test_Message", 'big')
c = pow(msg, e, n)
start = time()
m = pow(c, d, n)
print("Durée du déchiffrement :", round(time()-start, 3), "s")
print("Message clair :", m.to_bytes((m.bit_length()+7)//8, 'big'))
```

3 Exercice 3 – ElGamal

```
from random import randint

p = 0xFFFFFFFFB
g = 5

x = randint(2, p-2) # clé privée
y = pow(g, x, p)    # clé publique

m = 1234567
k = randint(2, p-2)
a = pow(g, k, p)
b = (m * pow(y, k, p)) % p

m_rec = (b * pow(a, p-1-x, p)) % p
print("Message récupéré :", m_rec == m)
```

4 Exercice 4 – Courbes elliptiques (ECDH)

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes
from Crypto.Cipher import AES

mohamed = ec.generate_private_key(ec.SECP256R1())
ali = ec.generate_private_key(ec.SECP256R1())

shared1 = mohamed.exchange(ec.ECDH(), ali.public_key())
shared2 = ali.exchange(ec.ECDH(), mohamed.public_key())
print("Clés identiques :", shared1 == shared2)

digest = hashes.Hash(hashes.SHA256())
digest.update(shared1)
aes_key = digest.finalize()

aes = AES.new(aes_key, AES.MODE_EAX)
c, tag = aes.encrypt_and_digest(b"Message ECC securise")
print("Chiffrement ECC+AES réussi")
```

5 Exercice 5 – Système Hybride RSA + AES

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
import secrets

key = RSA.generate(2048)
public_key = key.publickey()

aes_key = secrets.token_bytes(32)
data = b"Message long et confidentiel"

cipher_aes = AES.new(aes_key, AES.MODE_EAX)
ciphertext, tag = cipher_aes.encrypt_and_digest(data)

cipher_rsa = PKCS1_OAEP.new(public_key)
enc_aes_key = cipher_rsa.encrypt(aes_key)
print("Clé AES chiffrée (octets) :", len(enc_aes_key))
```

6 Exercice 6 – RSA Parity Oracle

```
from Crypto.PublicKey import RSA
from Crypto.Util.number import bytes_to_long
import base64

key = RSA.generate(1024)
n, e, d = key.n, key.e, key.d

msg = base64.b64decode(
    "
    VGhhdCdzIHdoeSBJIjZvdW5kIHlvdSBkb24ndCBwbGF5IGFyY3VuZCB3aXRoIHRobzSBGdW5reSBDb2
    =="
)
m = bytes_to_long(msg)
c = pow(m, e, n)

def oracle(cipher):
    return pow(cipher, d, n) % 2 == 0

print("Oracle(2*m) =", oracle((pow(2, e, n) * c) % n))
```

7 Exercice 8 – Faiblesses et controverses autour des courbes elliptiques (NSA)

Objectif : analyser les limites de confiance des paramètres cryptographiques et comprendre pourquoi certaines courbes elliptiques sont aujourd'hui déconseillées.

Contexte. Les algorithmes ECC (Elliptic Curve Cryptography) reposent sur la difficulté du problème du logarithme discret sur des courbes elliptiques. Cependant, la sécurité effective dépend fortement des *paramètres publics* choisis pour définir ces courbes.

Cas des courbes NIST. Les courbes standards NIST (P-256, P-384, P-521), historiquement recommandées par la NSA, ont été générées à partir de valeurs dites *seed* dont l'origine n'a jamais été expliquée publiquement. Cette absence de transparence a conduit à des soupçons de portes dérobées paramétriques, bien qu'aucune attaque mathématique publique n'ait été démontrée.

Révélation Dual_EC_DRBG. En 2013, les documents révélés par Edward Snowden ont montré que la NSA avait intentionnellement affaibli le générateur pseudo-aléatoire `Dual_EC_DRBG`, basé sur des courbes elliptiques, confirmant que des paramètres standards pouvaient être manipulés.

Conséquences pratiques. À la suite de ces révélations, de nombreux systèmes ont abandonné les courbes NIST au profit de courbes à paramètres transparents telles que `Curve25519` et `Ed25519`, aujourd'hui largement utilisées dans TLS, Signal et WireGuard.

Questions :

1. Pourquoi la transparence des paramètres est-elle cruciale en cryptographie ?
2. Pourquoi une backdoor paramétrique est-elle indétectable par des tests classiques ?
3. Quelle différence fondamentale existe-t-il entre P-256 et Curve25519 ?
4. Peut-on faire confiance à un algorithme sans faire confiance à ses paramètres ?